Variable Variables

This lesson introduces variable variables, also called dynamic variables, and discusses how you can use them in PHP.

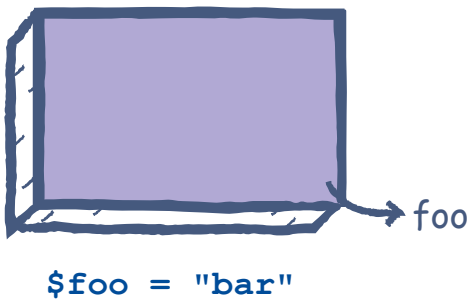
WE'LL COVER THE FOLLOWING

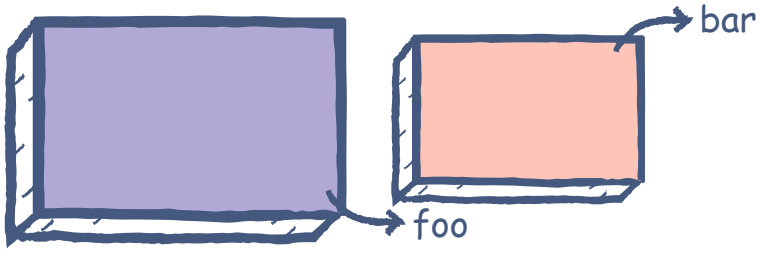
- What are Variable Variables?
- Implementation
- Quick Quiz

What are Variable Variables?

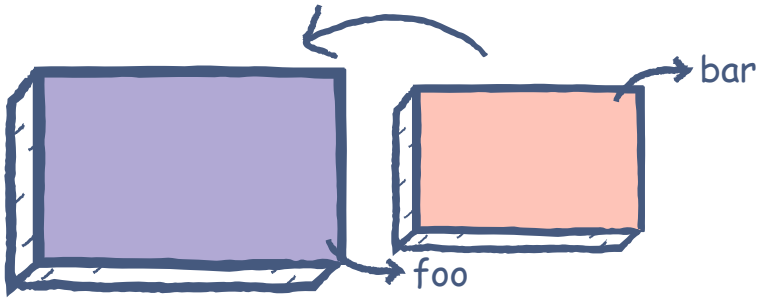
Using PHP, we can access data through dynamic variable names. The name of a variable can be stored in another variable, allowing it to be accessed dynamically. Such variables are known as *variable variables*.

To turn a variable into a *variable* variable, you put an extra `$` sign in front of your variable. This method is illustrated in the following figure:

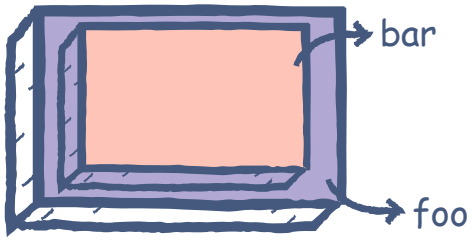




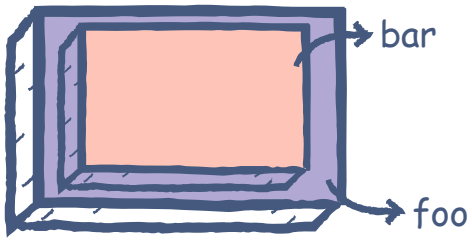

```
$foo = "bar"
```

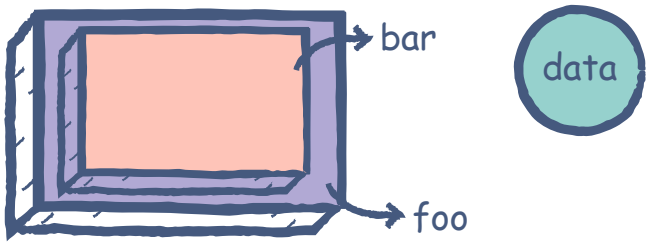

```
$foo = "bar"
```

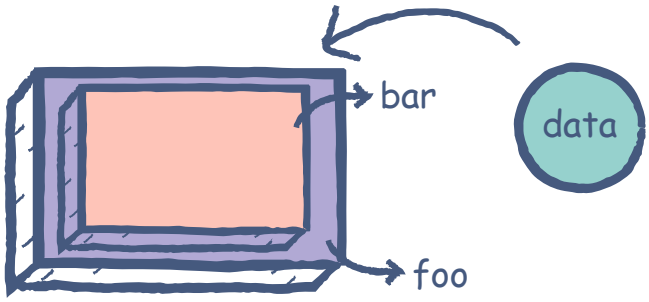

```
$foo = "bar"
```

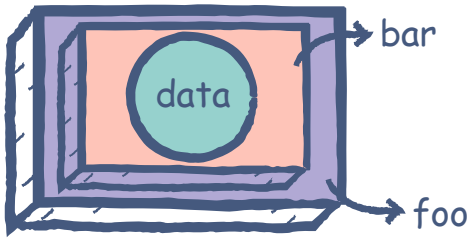

```
$$foo = "data"
```




```
$$foo = "data"
```

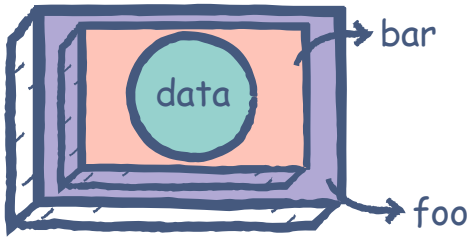




```
$$foo = "data"
```


```
$$foo = "data"
```

A variable containing a variable



Implementation

The following code snippet shows how this is done in PHP:

```
<?php
$foo = "bar"; // foo has value "bar"
$$foo = "data"; // bar has value "data"
echo "\$foo:\t";
echo $foo; //prints bar
echo "\n";

echo "\${\$foo}:\t";
echo ${$foo}; //prints data
echo "\n";

echo "\$\$foo:\t";
echo $$foo; //prints data
echo "\n";

echo "\$bar:\t";
echo $bar; //prints data
echo "\n";
?>
```

> **Note**: It is possible, but not required to put the variable name, say `$foo`, between {}.

The following examples are both equivalents and print the output “baz”:

```
<?php
$fooBar = 'baz';
$varPrefix = 'foo';

echo "\$fooBar:\t\t\t\t";
echo $fooBar; // Outputs "baz"
echo"\n";

echo "\${\$varPrefix . 'Bar'}:\t\t"; //varPrefix will give value foo and the dot operator cor
                                     //Putting $ before fooBar makes it a variable variable.
echo ${$varPrefix . 'Bar'}; // Also outputs "baz"
?>
```

Using {} is only mandatory when the name of the variable is itself an expression, like this:

```
<?php
${$variableNamePart1 . $variableNamePart2} = $value;
?>
```

It is nevertheless recommended to always use {} because it's more readable.

Quick Quiz

Q

What will the following code output?

```
<?php
$kangaroo = "Kanga";
$$kangaroo = "Roo";
echo "Hi, I'm ".$kangaroo." and this is ".$Kanga.".";
?>
```

COMPLETED 0%

1 of 1

Now that we've learnt about variables, let's move to PHP constants in the next lesson.

Precedence and Associativity

This lesson discusses two important characteristics of arithmetic operators, i.e., precedence and associativity.

WE'LL COVER THE FOLLOWING

- Precedence
- Associativity
  - Left Associativity
  - Right Associativity
- Operators with the same Precedence and Associativity
- Conclusion

Precedence

Operator precedence determines which operator is performed first in an expression with more than one operators. Operator precedence in PHP is similar to that of regular arithmetic operators.

- `*`, `/`, `%` operators have **equal** precedence.
- `+`, `-` operators have **equal** precedence.
- `*`, `/`, `%` have a **higher** precedence than `+`, `-`.

The operator with higher precedence is executed first in the expression.

Associativity

Associativity is used when two operators of the same precedence appear in an expression.

Left Associativity

Left associativity occurs when an expression is evaluated from left to right. For example `*` and `/` have the same precedence and their associativity is left

to right, so the expression

```
100 / 10 * 10
```

is treated as:

```
(100 / 10) * 10
```

Right Associativity

Right associativity occurs when an expression is evaluated from right to left. Like most programming languages, `=` and `print` in PHP have right associativity.

Run the following code to see a few examples:

```php
<?php
echo "5 - 3 + 2 = " . (5 - 3 + 2); // 5-3+2 is treated as (5-3)+2
echo "\n";
echo "5 + 3 * 2 = " . (5 + 3 * 2); // 5+3*2 is treated as 5+(3*2)
echo "\n";
echo "15 / 3 * 5 = " . (15 / 3 * 5); // 15/3*5 is treated as (15/3)*5
echo "\n";
echo "42 + 7 % 2 = " . (42 + 7 % 2); // 42+7%2 is treated as 42+(7%2)
?>
```

Operators with the same Precedence and Associativity

Naturally, you must be thinking, what will happen if there are two operators with the same precedence and associativity in an expression.

In this case, the expression is evaluated from left to right, i.e., it evaluates the operator that comes first (on the left).

For instance, `%` and `*` have the same precedence and associativity. Thus, given an expression `$a = 5 * 3 % 2;` the value of `$a` will be 1 because

$$(5 * 3)\%2 = 15\%2 = 1$$

And given an expression `$a = 5 % 3 * 2;` the value of `$a` will become 4

because

$$(5\%3) * 2 = (2 * 2) = 4$$

Run the code snippet below to see how this works:

```
<?php
$a = 5 * 3 % 2; // $a now is (5 * 3) % 2 => (15 % 2) => 1
echo $a . "\n";
$a = 5 % 3 * 2; // $a now is (5 % 3) * 2 => (2 * 2) => 4
echo $a . "\n";
?>
```

Conclusion

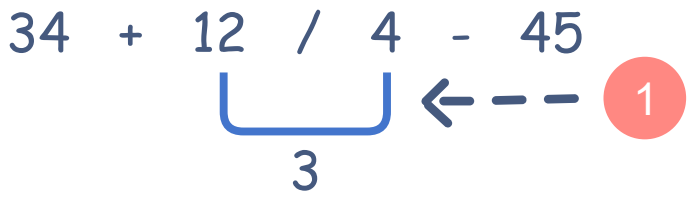
Thus, precedence and associativity are two characteristics of operators that determine the evaluation order of subexpressions in absence of brackets. This is illustrated in the figure below:

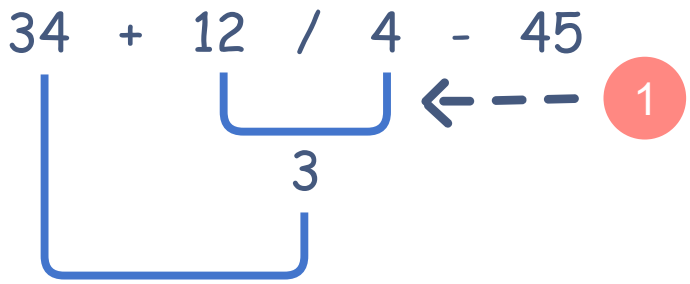
34 + 12 / 4 - 45

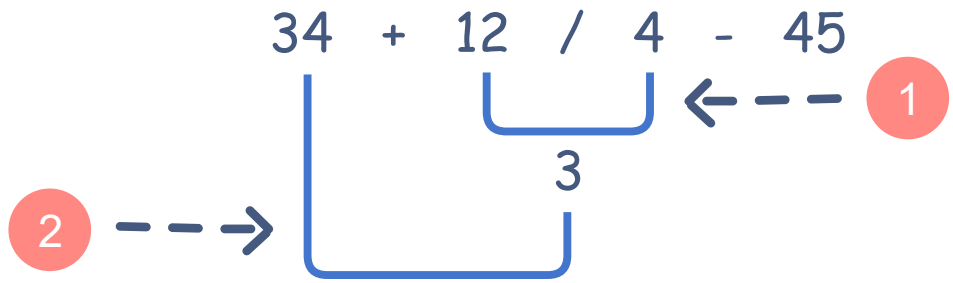
34 + 12 / 4 - 45

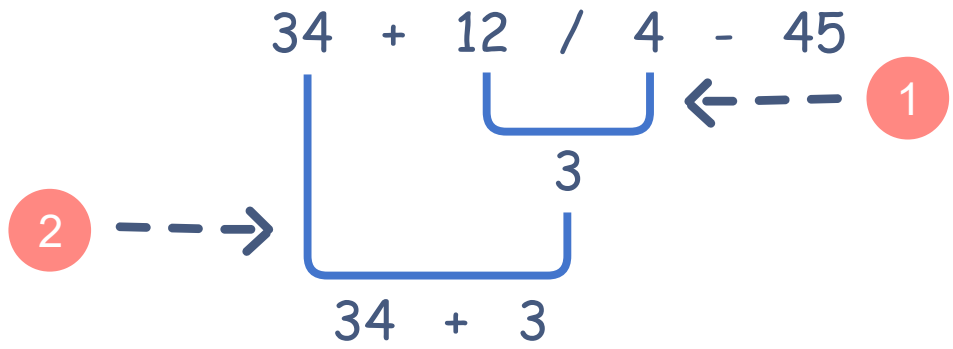
34 + 12 / 4 - 45

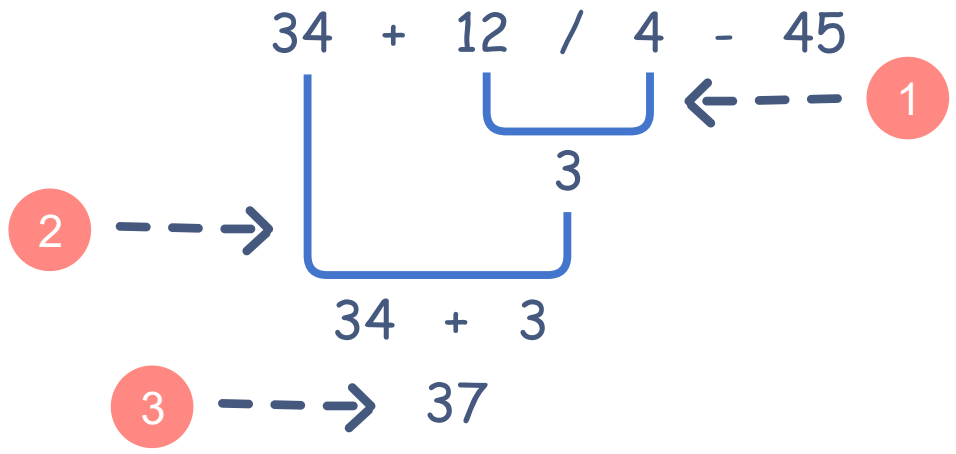
1

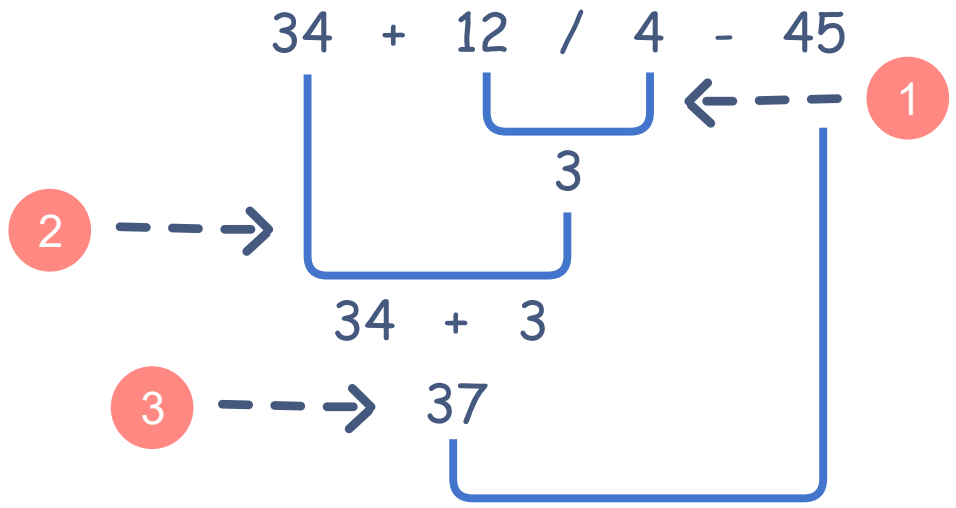

34 + 12 / 4 - 45
1
3

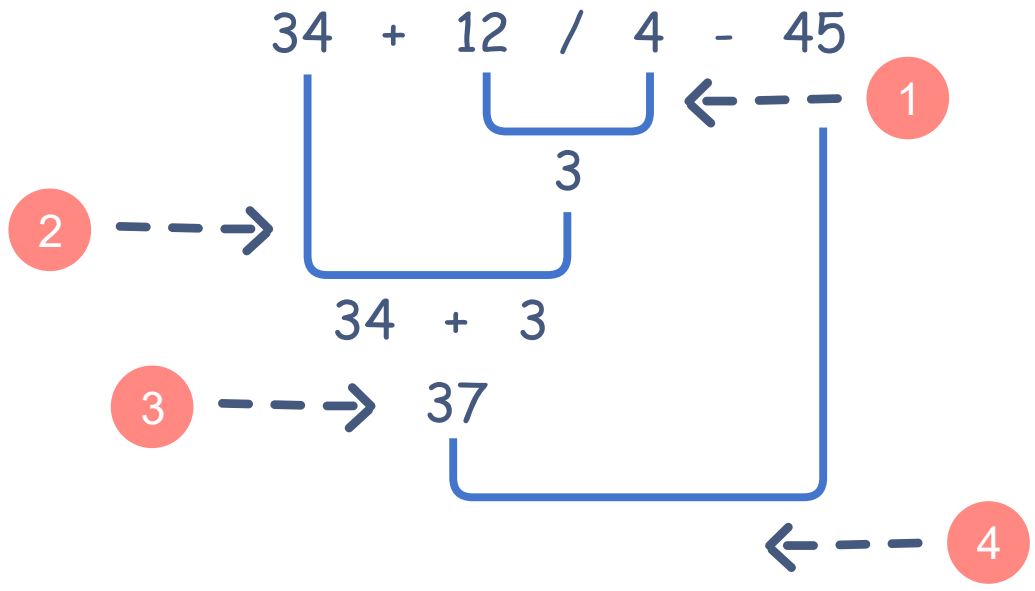

34 + 12 / 4 - 45
1
3

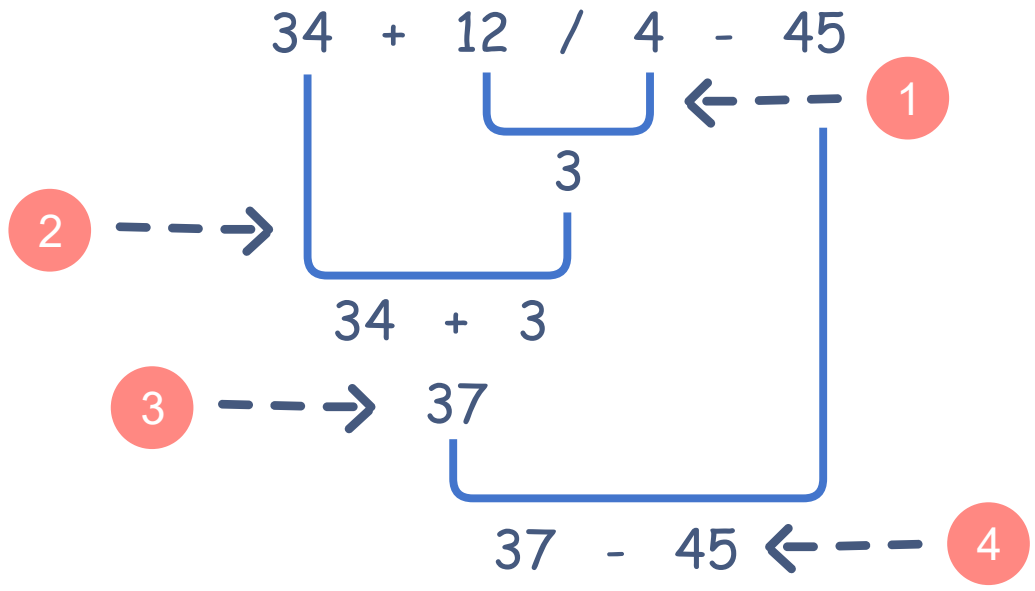

34 + 12 / 4 - 45
1
3
2

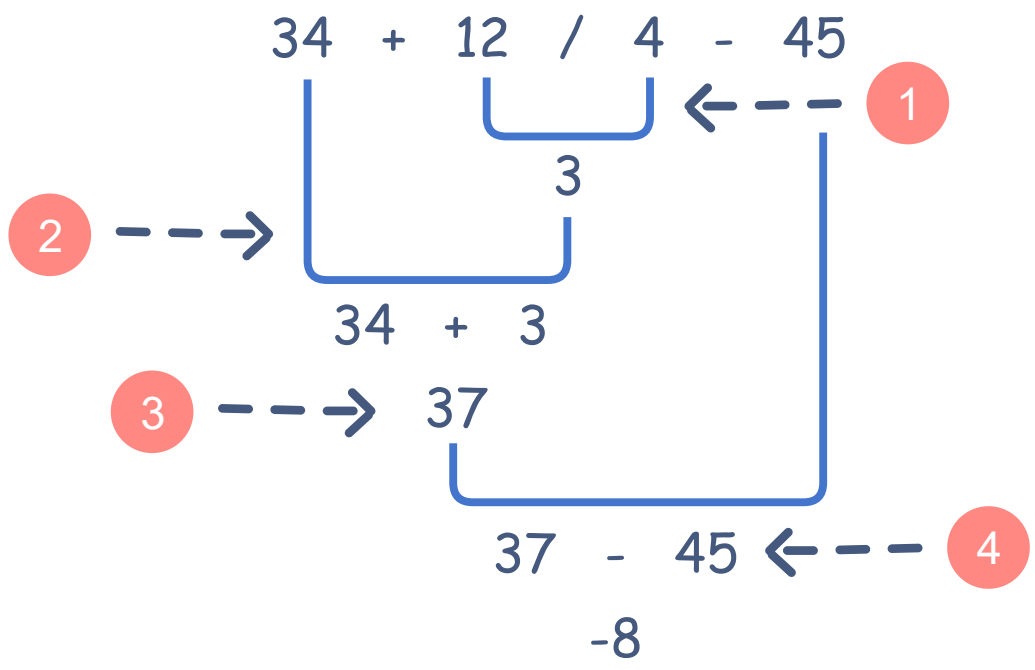

34 + 12 / 4 - 45
1
3
2
34 + 3

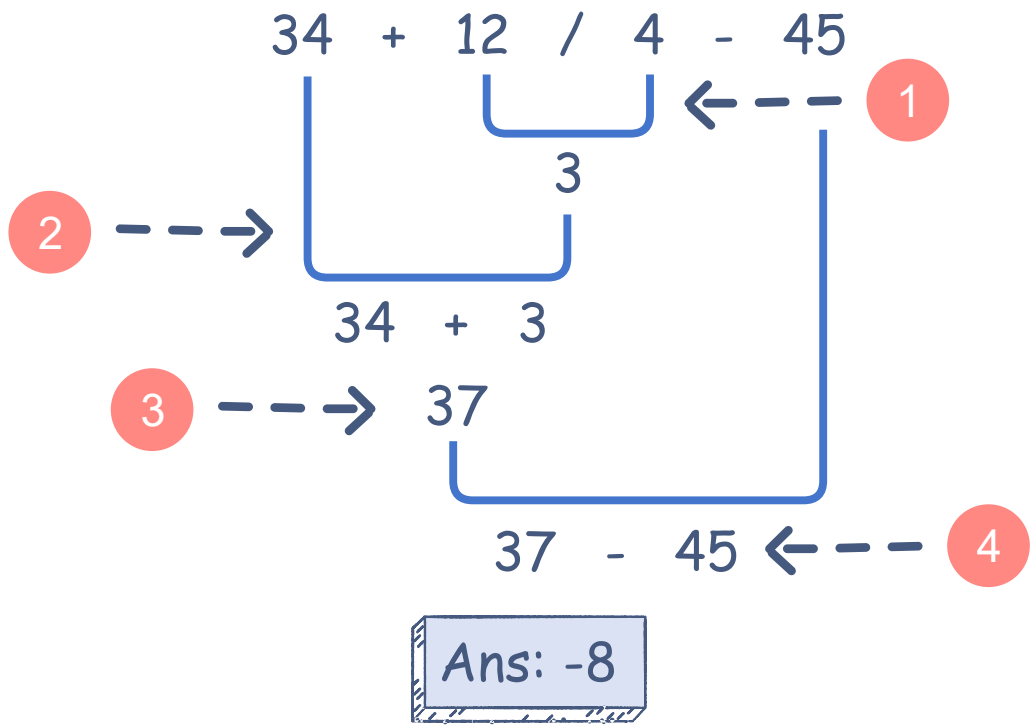

34 + 12 / 4 - 45
1
3
2
34 + 3
3
37


34 + 12 / 4 - 45
1
3
2
34 + 3
3
37


34 + 12 / 4 - 45
1
3
2
34 + 3
3
37
4


34 + 12 / 4 - 45
1
3
2
34 + 3
3
37
37 - 45
4





Remember: It is good to know precedence and associativity rules, but the best thing is to use brackets. Brackets increase readability of the code

> the best thing is to use brackets. Brackets increase readability of the code as the reader doesn't have to see the table to find out the order.

Now that you know all about arithmetic operations and how they behave, let's learn about another type of operators, the comparison operators.

Challenge 1: Find the Maximum Value

WE'LL COVER THE FOLLOWING

- Problem Statement
  - Input
  - Output
  - Sample Input & Output
- Coding Challenge

Problem Statement

Write a method named *findMaxVal()* to find the maximum integer value stored in an Array of any size.

Input

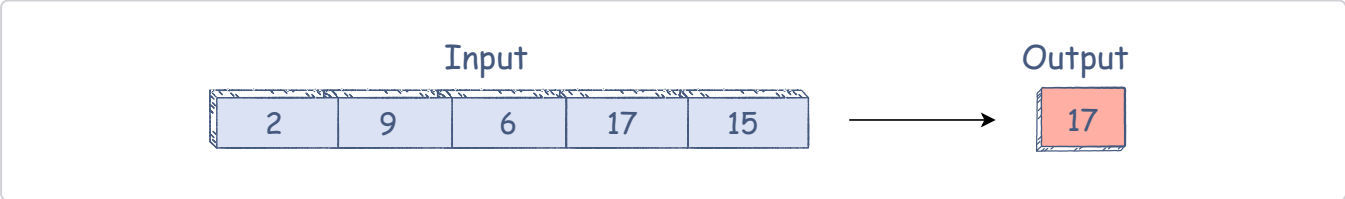
the array passed as a parameter.

Output

the maximum value found in the array as an integer

Sample Input & Output

Input: {2, 9, 6, 17, 15}
Output: 17



Coding Challenge

Think over the problem carefully and write your code below. It's recommended that you try solving the problem by yourself before referring to the solution part.

Good luck!

```
<?php
//Returns maximum value from Array passed as parameter
function findMaxVal($arr)
{

  //write your code here
  return -1; //change this line to return the maximum value

}
?>
```

Let's move on to a more complex form of arrays, the multi-dimensional arrays in the next lesson.

$this and self

This lesson discusses the use of this and self in accessing members in classes.

WE'LL COVER THE FOLLOWING

- $this
  - The Object Operator
- self
  - The Scope Resolution Operator
- this versus self
- Example

$this

The pseudo-variable `$this` refers to a member of a class according to an instance of that class. It provides a reference to the calling object, that is the object to which the method or property belongs to. Note that the member of the class must be non-static.

The Object Operator

The `->` symbol is a built-in construct in PHP that is used with the `$this` keyword to access contained **methods** and **properties**.

self

In PHP, like many other languages, the `self` keyword refers to properties and methods inside the scope of a class. This pseudo-variable provides a reference to the calling object, that is the object to which the method or property belongs to.

The Scope Resolution Operator

The `::` symbol, known as the *scope resolution operator* is an in-built construct

in PHP that is used to access contained **methods** and **properties**. It is used with the `self` keyword.

> ### this versus self
>
> - Use `$this->member` for accessing *non-static* members (methods and properties).
> - Use `self::$member` for accessing static members (methods and properties).

Example

Run the code below to see how you can use `$this` and `self` in PHP.

```
<?php
class Circle
{
    // properties
    public $radius= 0; //declaring public member
    public static $pi=3.14;  //declaring a public static member

    // Method to get the Circumference
    public function getCircumference(){
        return (2 * self::$pi * $this->radius );
    }

    // Method to get the area
    public function getArea(){
        return ($this->radius * $this->radius*self::$pi);
    }

    // Method to get the diameter
    public function getDiameter(){
        return ($this->radius * 2);
    }
}

// Create a new Circle class object
$obj = new Circle;


// Set object properties values
$obj->radius = 4;


// Read the object properties values again to show the change
echo "Radius is ". $obj->radius . "\n";
echo "Diamater is ". $obj->getDiameter() . "\n";
```

```
// Call the object methods
echo "Circumference is ". $obj->getCircumference(),"\n";
echo "Area is " .$obj->getArea()."\n";

echo "Value of pi is " .$obj::$pi;
?>
```

As you can you see in the code above, when accessing *non static* members like `radius` we are using `$this` and `->` to access the value, whereas, when accessing the *static* member like `pi` we used `self` and `::` to access the value.

In the next lesson we'll discuss *constructors* and *destructors* as well as their use in classes.

Constructors & Destructors

This lesson introduces the concept of constructors, destructors, how to declare and call them.

WE'LL COVER THE FOLLOWING

- Introduction
- Constructor Declaration
- Constructor Definition
  - Calling a constructor
- Introduction to Destructors

Introduction

A *constructor* is used to initialize *member* variables when an *object* is declared. It is automatically called at the time when the *object* of the *class* is declared.

> **Note:** A *constructor* is a *member* function that is usually `public`.

Also keep in mind that unlike other *methods* defined inside a class, a constructor **cannot** *return* a value.

Constructor Declaration

Classes can define a special `__construct()` *method*, which is executed as part of object creation.

Let's take a look at how it is defined:

```
<?php
```

```
class Shape {

public $sides = 0;

public $name = " ";

 //initializes $name to $Name and $sides to $Sides
public function __construct($Name, $Sides)

?>
```

Constructor Definition

The constructor for `Shape` can be defined as follows:

```
<?php
class Shape
{

    public $sides = 0;

    public $name = " ";

    public function __construct($name, $sides)
    { //defining a constructor
        $this->sides = $sides; //initializing $this->sides to $sides
        $this->name = $name; //initializing $this->name to $name

    }

    public function description()
    { //method to display name and sides of a shape
        echo "A $this->name with $this->sides sides.";
    }

}

$myShape = new Shape("hexagon", 6); //making an object and passing values to the constructor
$myShape->description(); // A shape with 6 sides

?>
```

Calling a constructor

As you can see above in **line 21**, the way to call a *constructor* is not like a normal *member* function.

- It is called in *object* declaration.
- It creates a `Shape` object.

- Then *calls* the *constructor* to initialize *variables*.

In the example, the **constructor** takes as parameters the *name* and *number of sides* and sets their values.

Introduction to Destructors

Destructors are the opposite of **constructors**, as they define the *final* behavior of an *object* and execute when the object is no longer in use.

An object's **destructor**, which takes **no** *parameters*, is called sometime after an *object* is no longer *referenced*, but the complexities of *garbage* collection make the specific timing of *destructors* uncertain.

> **Note: Destructors** are not called but are invoked *automatically*.

```
<?php
class Shape
{

    public $sides = 0;

    public $name = " ";

    public function __construct($name, $sides)
    { //defining a constructor
        $this->sides = $sides; //initializing $this->sides to $sides
        $this->name = $name; //initializing $this->name to $name

    }

    public function __destruct()
    { //destructor for Shape gets called at the end
        echo "Destructor Called!\n";
    }

    public function description()
    { //method to display name and sides of a shape
        echo "A $this->name with $this->sides sides.\n";
    }

}

$myShape = new Shape("hexagon", 6); //making an object and passing values to the constructor
$myShape->description(); // A shape with 6 sides
```

This marks the end of our discussion on **constructors** and **destructors**. In the next lesson, we will discuss different access modifiers used while declaring member variables in classes.

Method and Property Visibility

This lesson discusses access modifiers like public, private and protected and how they affect the visibility of members.

WE'LL COVER THE FOLLOWING

- Public
  - Example
- Private
  - Example
- Protected
  - Example

Access modifiers provide access to the variables of a class. In this lesson, we will discuss the **three** visibility types that you can apply to **methods** (*class/object functions*) and **properties** (*class/object variables*) within a class. Access modifiers provide access control for the *method* or *property* to which they are applied.

Public

Declaring a method or a property as `public` allows the method or property to be accessed by:

- The class that declared it.
- The classes that inherits from the declared class.
- Any external objects, classes, or code outside the class hierarchy.

Example

Run the code below to see how the `public` keyword works:

```
<?php
```

```
class Car
{
    public $name = " ";

    public function display()
    {
        echo "Name: $this->name" . "\n";
    }

    public function __construct($name)
    {
        $this->name = $name;

    }
}

$obj1 = new Car("BMW"); //creating an object of car and setting its name as BMW
echo "Name: " . $obj1->name; //accessing the "name" property of obj1 directly outside of clas
echo "\n";

$obj2 = new Car("Mercedes"); //creating an object of car and setting its name as Mercedes
echo $obj2->display(); //accessing the "display" method of obj1 directly outside of class
?>
```

The example above shows that the `public` *property*, `$name` and the `public` *method* `display()`, can be accessed outside directly by the objects of the class.

Private

Declaring a method or a property as `private` allows the method or property to be accessed by:

- **Only** the class that declared it.

A `private` *method* or *property* is only visible and accessible within the class that created it.

Example

Run the code below to see how the `private` keyword works:

```
<?php
class Car
{
    public $name = " ";
    private $plateNumber;

    public function display()
    {
```

```
    {
        echo "Name: $this->name" . "\n";
    }

    public function setPlateNumber($number)
    { //sets value of property plateNumber
        $this->plateNumber = $number;
    }

    public function getPlateNumber()
    { //returns the property "plateNumber"
        return $this->plateNumber;
    }

    public function __construct($name, $number)
    {
        $this->name = $name;
        $this->plateNumber = $number;

    }
}

$obj1 = new Car("BMW", 68775); //making a car object with values of name and platenumber set
echo $obj1->display(); //displaying name of car
echo "Plate number: " . $obj1->getPlateNumber(); //accessing plateNumber by calling getPlateN
echo "\n";
$obj1->setPlateNumber(47798); //changing PlateNumber value using setPlateNumber
echo "Plate number: " . $obj1->getPlateNumber(); //accessing plateNumber by calling getPlateN
echo "\n";

$obj2 = new Car("Mercedes", 89976);
//uncomment the line below and try running the code
//you will get an error as you cannot directly access a private member outside of the class i
//echo $obj2->plateNumber;
?>
```

The example above shows that the methods `display()`, `getPlateNumber()` and `setPlateNumber()` in class `Car` can access the `private` property `$plateNumber`. The get and set functions are used to access as well as set values of the **private** members in a class.

In the code above, when you run the code accessing the `private` member `$plateNumber` directly, an error of *undefined property* is thrown as private property is accessed out of its scope.

Protected

Declaring a method or a property as `protected` allows the method or property to be accessed by:

- the class that declared it
- the classes that inherits from the declared class

This **does not allow** external objects, classes, or code outside the class hierarchy to access these methods or properties. An error occurs if they try to access it.

Example

Run the code below to see how the `protected` keyword works:

```
<?php
class Car
{
    public $name = " ";
    protected $power = 2500;

    public function display()
    {
        echo "Name: $this->name" . "\n";
    }

    public function __construct($name)
    {
        $this->name = $name;

    }
}

$obj = new Car("Blue");
echo $obj->display();
echo $obj->power; //comment out this line to prevent an error
?>
```

The example above shows that when the main script tries to access the `protected` variable `$power`, an error: *cannot access protected property* is thrown.

> Removing `line 20`, in the above code, will allow the code to run successfully.

The table below illustrates the difference between all three access modifiers:

Modifier	Class	Subclass	World
Private	✓	✗	✗
Protected	✓	✓	✗
Public	✓	✓	✓

> **Note:** Objects of the same class will have access to each other's `private` and `protected` members even though they are not the same instances.

Run the code below to see an example:

```
<?php

class Test
{
        private $privateM=1;
        protected $protectedM=2;

        public function increase(Test $test)
        {
                $test-> privateM *= 10;
                $test-> protectedM *= 10;
        }

        public function __toString()
        {
                return "privateMember = ".$this->privateM.", protectedMember = ".$this->prote
        }
}

$test1 = new Test();
$test2 = new Test();
echo "before test1: $test1\n";

//call $test2 method on another instance of the Test class - $test1
$test2->increase($test1);

echo "after test1: $test1\n";
?>
```

This marks the end of our discussion on *classes*. In the next lesson we'll discuss the concept of *inheritance*.

Inheritance

This lesson introduces the concept of Inheritance focusing on base and derived classes.

WE'LL COVER THE FOLLOWING

- What is Inheritance
- Terminology
- Characteristics
- Notation
- Example

What is Inheritance

Inheritance is a very useful and popular concept that enhances the object-oriented programming experience. It allows a programmer to create and define classes that can inherit and build upon functionalities already present in existing classes without having to duplicate a lot of the code.

- Provides a way to create a **new** class from an **existing** class.
- The **new** class is a *specialized* version of the **existing** class.
- Allows the **new** class to override or redefine inherited *methods* from the **existing** class to perform differently than how they are defined in the **existing** class.

Terminology

- **Base Class** (or Parent): *inherited* by **child** class.
- **Derived Class** (or child): *inherits* from base class.

Characteristics

A **derived** class has:

- All *members* defined in the **derived** class.
- All *members* declared in the **base** class.

A **derived** class can:

- Use all `public` and `protected` members defined in the **derived** class.
- Use all `public` and `protected` members defined in the **base** class.
- Override an *inherited* member

A **derived** class cannot:

- *Inherit constructors* and **destructors**

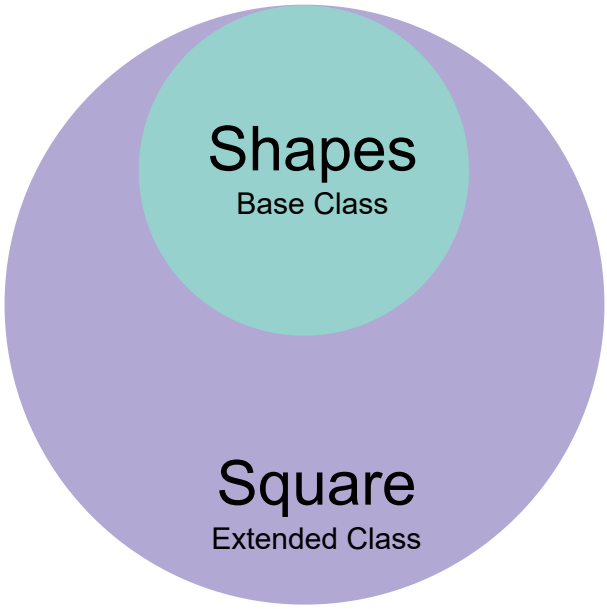
Inheritance is one of the major reasons for using object-oriented programming in PHP.

Notation

Classes can *inherit* the properties and methods of another class using the keyword `extends`.

Let's take a look at the notation below.

```
class Shape { //base class
 //body
}

//derived class inherits from another class by using the keyword extends
class Square extends Shape {
      //body
}
```



Example

Let's consider an example with the *base* class `Shape` and the *derived* class `Square`.

```
<?php
class Shape
{ //base class
    public $sides = 0;

    public $name = " ";

    public function __construct($name, $sides)
    { //base class constructor
        $this->sides = $sides;
        $this->name = $name;
    }

    public function description()
    {
        return "A $this->name with $this->sides sides.";
    }
}

class Square extends Shape
{ //derived class inheriting from base class
    public $sideLength = 0;

    public function __construct($sideLength)
    {
        parent::__construct("square", 4); //calling parent class constructor
        $this->sideLength = $sideLength;
```

```
    }

    public function perimeter()

    {
        return $this->sides * $this->sideLength;
    }

    public function area()
    {
        return $this->sideLength * $this->sideLength;
    }
}

$mySquare = new Square(10);
$mySquare->description();
echo "Perimeter of the square is " . $mySquare->perimeter() . "\n";
echo "Area of the square is ";
echo $mySquare->area();
?>
```

In the above example, the **extended class** `Square` *extends* from the **base class**, `Shape`. While doing so, it is inheriting all of its properties, `$sides` and `$names`, and its methods, `description()`.

It then goes on to define new properties: `$sideLength` and new methods: `perimeter()` and `area()`.

As seen in the example above, a derived class cannot inherit the base class constructor however it can call the base class constructor as seen in **line 25**. Hence, once the derived class constructor is called it will further call upon the parent class constructor, where it'll set the `name` and `sides`, and after that, it'll set the `sideLength` as well.

This marks the end of our discussion on inheritance. In the next chapter, we will discuss *exception handling*.

Quiz

A short quiz to test your understanding of classes.

1

The `new` keyword can be used to create an object of a Class?

2

The variable f below represents

```
<?php

class Foo {
public $x = 100;
}

$f = new Foo;
echo "x is: ".$f->x;


?>
```

3

`$this->member` for accessing static members (methods and properties).

4

A destructor needs to be called at the end of the program.

5

What is the error in the code below?

```
<?php

class Dog
{

    private $age= 3;
    private static $name= "Ruffy";
```

```
    public function getAge(){

        return $this->age;
    }

    public function getName(){
            return $this->name;
    }
 }


$obj = new Dog;

echo "Name is ". $obj->getName();
echo "Age is ". $obj->getAge();


?>
```

6

Declaring a method or a property as protected allows external objects outside the class to access these methods or properties.

7

There is no difference in a protected and public variable of a mother class in terms of accessibility and use for a derived class.

Is this statement correct?

8

A derived class can change the definition of an inherited member.

Check Answers

Exercise 1: Calculating Area

This is a basic exercise on defining a class that helps calculate the area of a triangle.

WE'LL COVER THE FOLLOWING

- Problem Statement
  - Input
  - Output
  - Sample Input
  - Sample Output
- Coding Challenge

Problem Statement

Let's start with a very basic example.

Write a *class* called `Triangle` having **two** `public` variables for `length` and `height` and **one** *member* function called `area` which will *return* the **area** of the right angle **triangle**. You can calculate the area of a triangle using

$$Area = 1/2(length)(height)$$

Input

two variables (integer or float) as the dimensions of the triangle

Output

area of that triangle

Sample Input

4, 5

Sample Output

Coding Challenge

Write your code below. It is recommended that you try solving the exercise yourself before viewing the solution.

Good Luck!

> Note: There is a `test` function given in the code for testing purposes. Do not modify it.

```
<?php

//define your class here
// name your class Triangle
//name your function area



//This is a test function. Do not change it.
function test($length, $height) {

  $answer=0;
  $obj = new Triangle;
  $obj->length= $length;
  $obj->height= $height;
  echo "Area: ".$obj->area();
  $answer = $obj->area();

  return $answer;
}
?>
```

Exercise 2: Students Average Marks

Calculate a student's average marks using the concept of classes.

WE'LL COVER THE FOLLOWING

- Problem Statement
- Coding Challenge

Problem Statement

Write a **class** called `student` with

- `private` *member variables*:
  - `name`,
  - `mark1` and `mark2`
- And *methods*:
  - `getMarks($marknumber)`, a *method* which should return `mark1` if `marknumber` equals **1** and `mark2` otherwise.
  - `calcAverage()` *method* should take the **two** marks entered and *return* their **average**.

Also *define*

- A *constructor* that takes the **three** `private` *variables* and *sets* them to given *values*.

Coding Challenge

Write your code below. It is recommended that you try solving the exercise yourself before viewing the solution.

Good Luck!

```php
<?php
class Student
{

    //define private variables here


    public function __construct($na, $ma1, $ma2)
    {
        //write definition here

    }
    public function getMarks($marknumber)
    {
        //write definition here
        return -14567; //return correct answer here

    }
    public function calcAverage()
    {
        //write definition here
        return -14567; //return correct answer here

    }
}
?>
```

PHP Exceptions

This lesson discusses what exceptions are and how PHP allows you to handle them.

WE'LL COVER THE FOLLOWING

- What is an Exception?
- What Happens When an Exception is Triggered?
- Exception Handling in PHP

What is an Exception?

An **exception** is an unexpected program result that can be handled by the program itself. **Exception Handling** in PHP is almost similar to exception handling in all programming languages.

What Happens When an Exception is Triggered?

When an exception is triggered, the current code state is saved. The code execution will switch to a predefined (custom) exception handler function Depending on the situation, the handler may then resume the execution from the saved code state, terminate the script execution or continue the script from a different location in the code.

Exception Handling in PHP

PHP provides the following specialized keywords for this purpose:

- try
- throw
- catch
- finally

In the next lesson, we will learn about these keywords in detail.

The Exception Class

This lesson discusses how you can use the Exception class for exception handling using various coding scenarios.

WE'LL COVER THE FOLLOWING

- Components
- Methods

PHP has a class-based exception handling mechanism. The `Exception` class is a built-in class with various methods and properties. In this lesson, we will be just be concerned with learning the basics of exception handling using the PHP `Exception` class.

Components

- `try`: It is the block of code in which exception may arise.
- `catch`: It is the block of code that will be executed when a particular exception is *thrown*.
- `throw`: It is used to *throw* an exception. It is also used to list the exceptions that a function throws, but doesn't handle itself.
- `finally`: This is the block of code that executes at the end once the exception is thrown and/or handled.

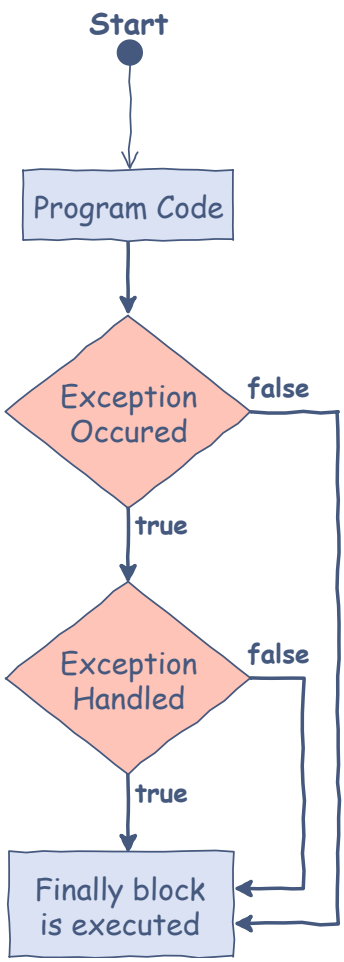
Run the code below to see how exception handling works:

```
<?php
function distance($speed, $time){

    if($time <= 0){
        throw new Exception('Time cannot be zero or negative.'); // Throw exception if time i
    } else{

        $d = $speed*$time;
        echo "$speed * $time = $d";
```

```
    }
}


try{
    distance(10,2);
    distance(30,-4); //code will stop execution at this point (due to negative time) and star
    distance(15,3);

    echo 'All calculations done!'; // If an exception is thrown, this line will not execute
  }

// catch block is executed when an exception is thrown in the try block
// an object $e of Exception class is created
catch(Exception $e){

    echo "\n". "Caught exception: " . $e->getMessage(); //Exception handling
}

echo "\n"."Hello World!"; // Continue execution
?>
```

> **Note:** The code will **terminate** if there is **no** catch for a thrown exception. So if you want the script to continue executing beyond the point where the exception has occurred, you must have at least **one** corresponding catch block for each try block.

The following figure illustrates the flow of a program with exception handling:



Methods

PHP's `Exception` class also provides the following methods for detailed information:

- `getCode()`
- `getFile()`
- `getLine()`
- `getTraceAsString()`

Other methods and properties of the Exception class can be seen in the PHP documentation.

php without_catch | php with_catch

```php
<?php
function division($a, $b){

    if($b ==0){
        throw new Exception('Divisor is zero'); // Throw exception if divisor is zero
    } else{
```

```
    } else{

        $c = $a/$b;

        echo "$a / $b = $c";
    }
}

try{
    division(10, 2);
    division(15, 0);
    division(30, -4);


    echo 'All calculations done!';// If an exception is thrown, this line will not execute
  }

// write your catch statement here

echo "\n"."Hello World!"; // Continue execution
?>
```

The code above does not work since there is no catch statement in it.

Can you write a catch statement of your own on `line 22` in the code and see if it runs? If you can't write it just yet, don't fret; see the code tab on the right.

In the next lesson, we will learn how custom exceptions can be used in PHP.

Custom Exceptions

Create custom exceptions in PHP that are tailor-made for your program.

WE'LL COVER THE FOLLOWING

- Defining Custom Exceptions
- Sample Code for Custom Exceptions

Defining Custom Exceptions

PHP provides the functionality to create custom exception handlers. It allows you to give separate exception block for each type of exception.

The class must be an **extension** of the built-in `Exception` class since it is the base class. The custom exception class *inherits* all the objects, properties and methods, from PHP's `Exception` class. You can also add custom objects to this new *extended* exception class.

Sample Code for Custom Exceptions

Following is the example of a custom exception class:

```
<?php
  class DecelerationException extends Exception{} //DecelerationException inherits Exception
  class TimeException extends Exception{} //TimeException inherits Exception

function acceleration($finalSpeed,$initialSpeed,$time){

  if($time <= 0){
    throw new TimeException('Time cannot be negative or zero.'); // Throw exception if time i
  }
  if($initialSpeed > $finalSpeed){
    throw new DecelerationException('It is deceleration.'); // Throw exception if initial spe
  }
  else{
    $a = ($finalSpeed-$initialSpeed)/$time;
    echo "($finalSpeed-$initialSpeed)/$time = $a";
  }
}
```

```
try{
  acceleration(20,10, 2);
  acceleration(30,10, -4); //code will stop execution at this point and start finding the cat

  acceleration(15,20, 5); //$initialSpeed>$finalSpeed

  echo 'All calculations done!';// If an exception is thrown, this line will not execute
}

catch(DecelerationException $e){

  echo "\n". "Caught deceleration exception: " . $e->getMessage(); //Exception handling
}
catch(TimeException $e){

  echo "\n". "Caught time exception: " . $e->getMessage(); //Exception handling
}

echo "\n"."Hello World!"; // Continue execution
?>
```

Exchange positions of `line 20` and `line 21` to see which exception is caught now.

This was all about exception handling in PHP. Now you can handle run-time anomalies or abnormal conditions that a program encounters during its execution. Let's solve a quick quiz in the next lesson.